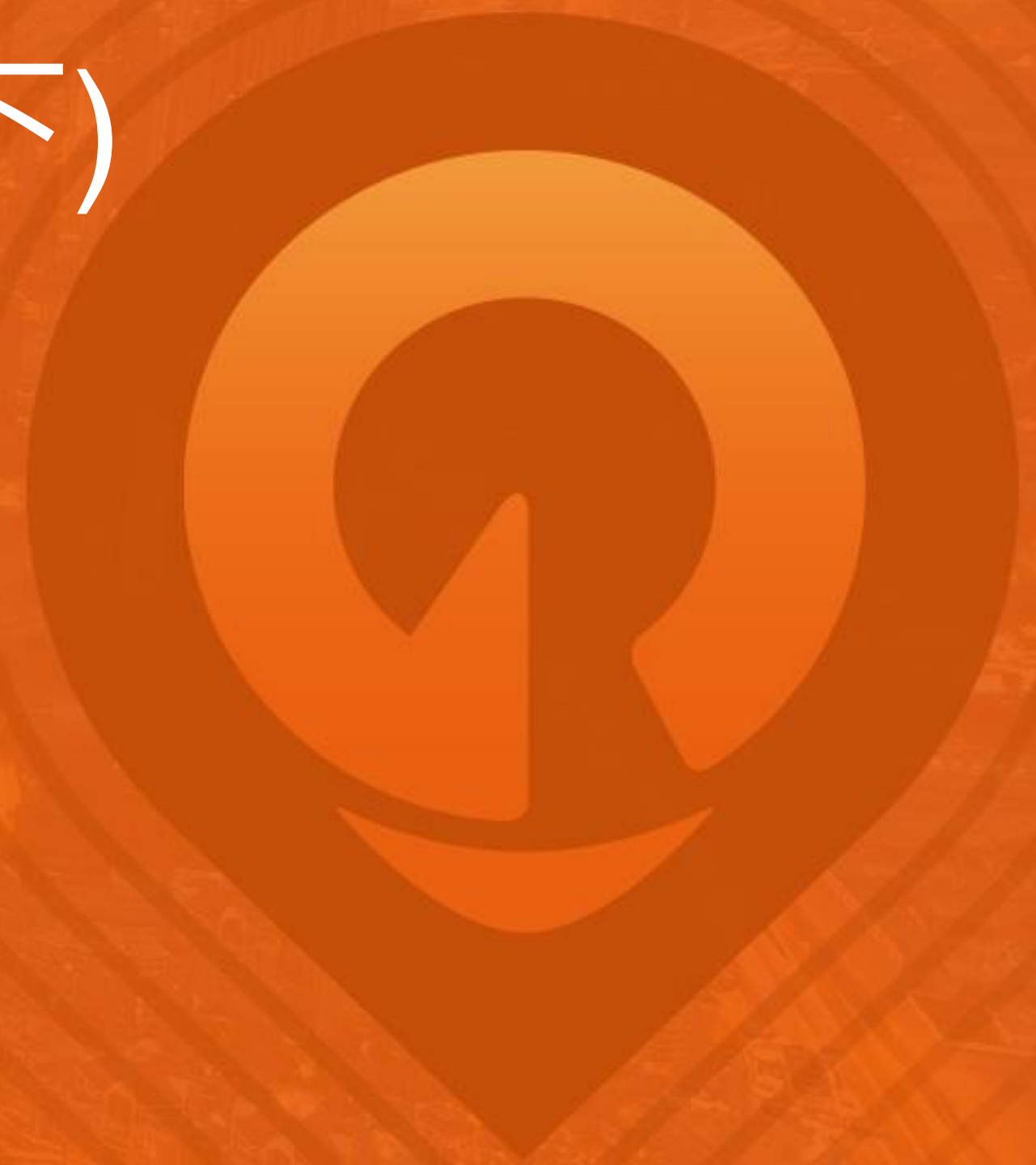


大模型应用开发框架 LangChain (下)

彭靖田

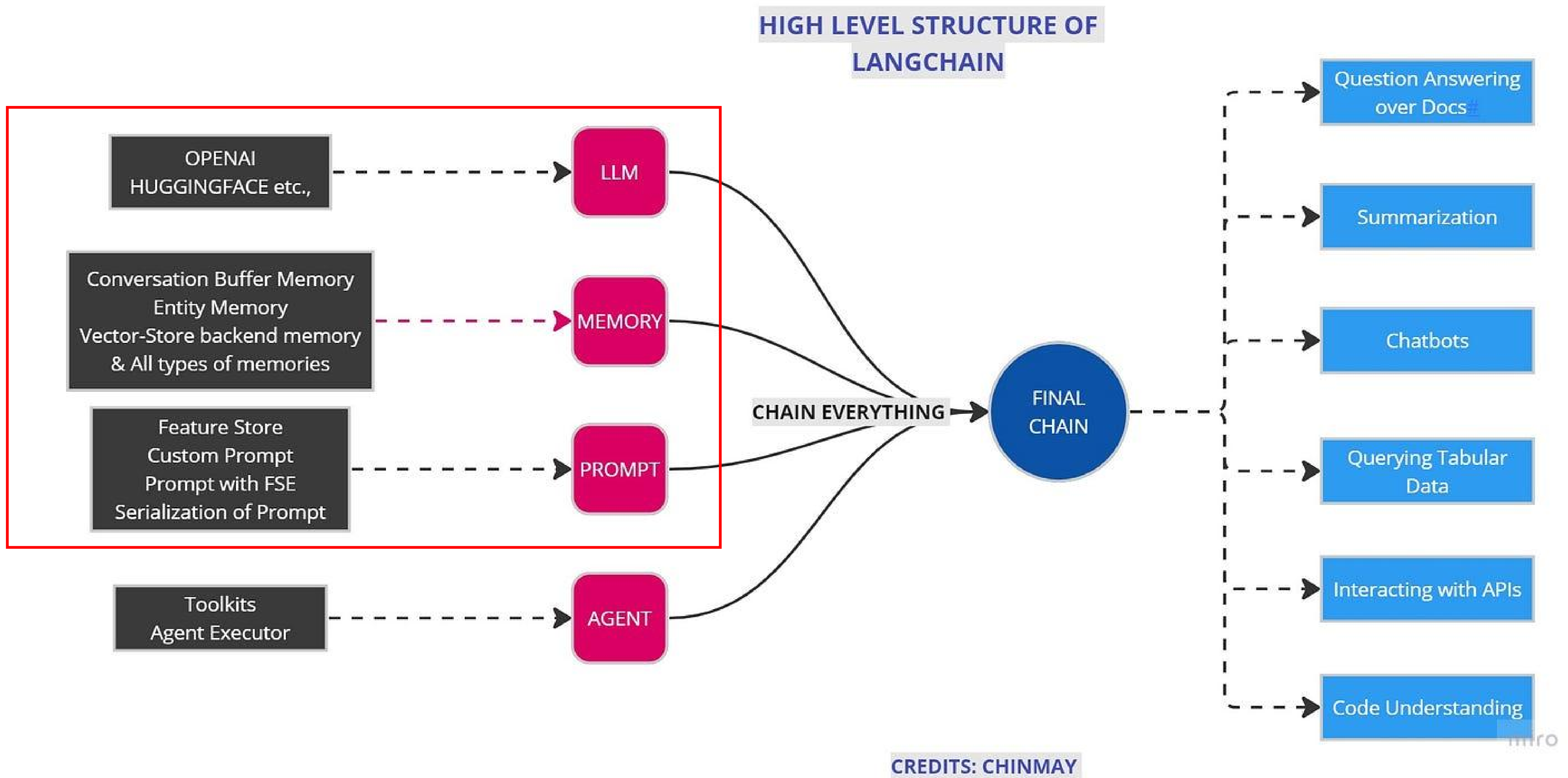
Google Developer Experts
LangChain Developer



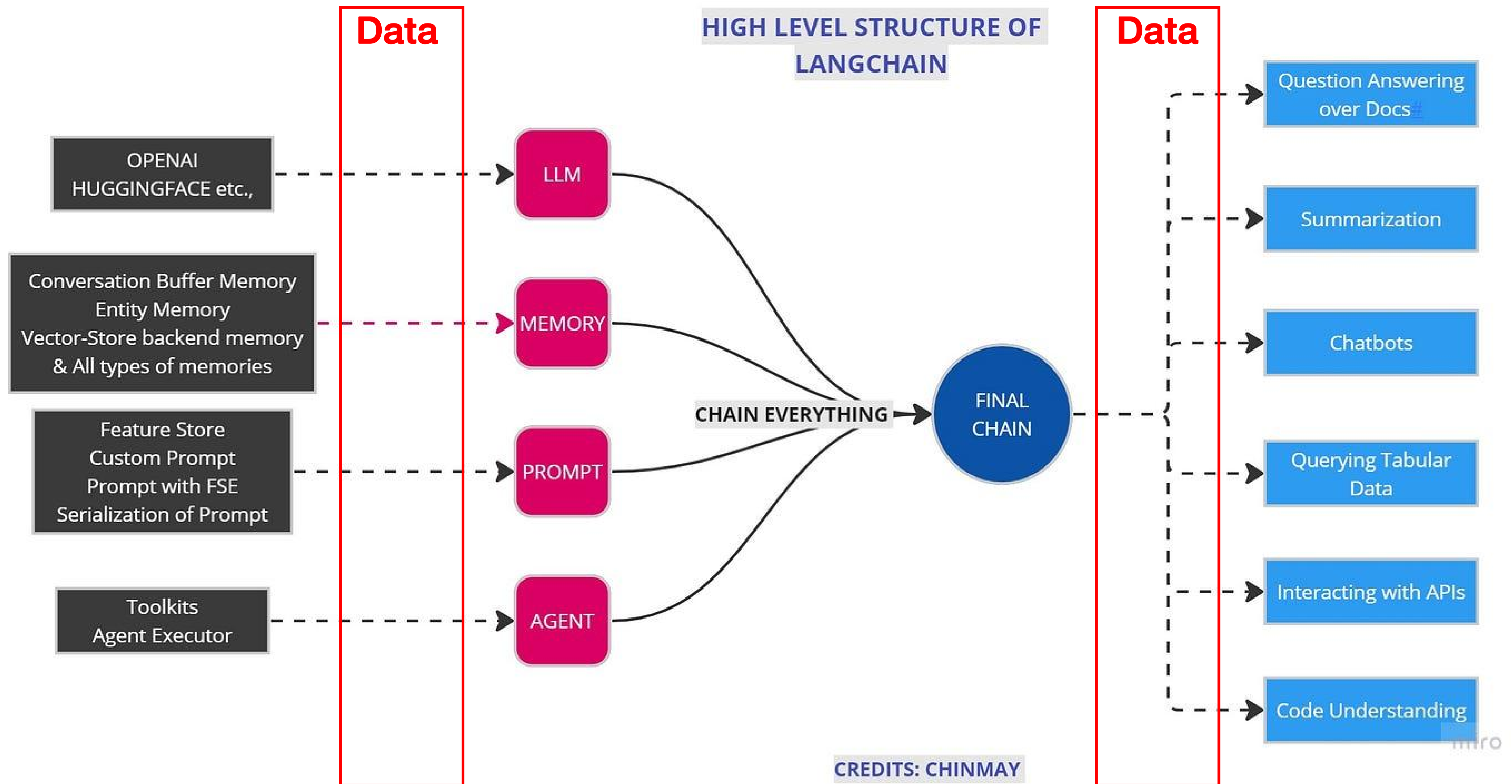
目录

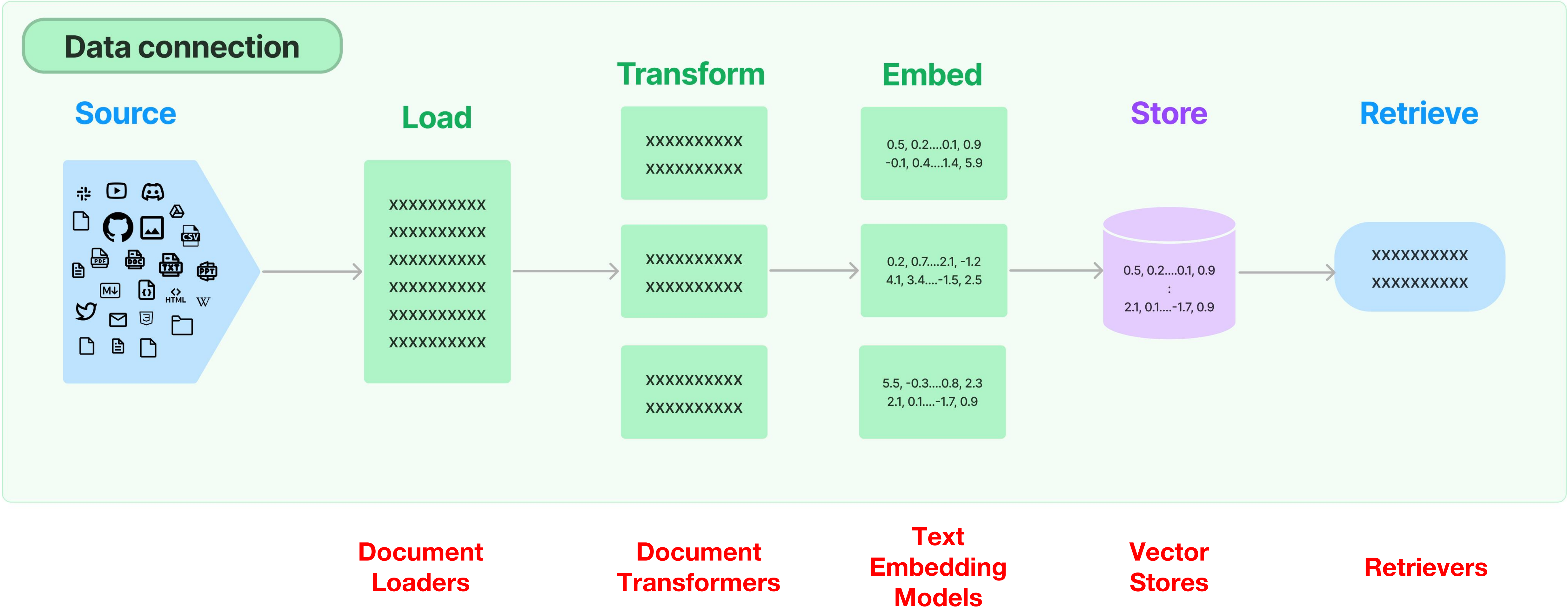
- 框架原生的数据处理流 Data Connection
 - 文档加载器 (Document Loaders)
 - 文档转换器 (Document Transformers)
 - 文本向量模型 (Text Embedding Models)
 - 向量数据库 (Vector Stores)
 - 检索器 (Retrievers)
- 构建复杂应用的代理系统 Agents
 - Agent 理论基础: ReAct
 - LLM 推理能力: CoT, ToT
 - LLM 操作能力: WebGPT, SayCan
 - LangChain Agents 模块设计与原理剖析
 - Module: Agent, Tools, Toolkits,
 - Runtime: AgentExecutor, PlanAndExecute , AutoGPT,
 - 上手第一个Agent: Google Search + LLM
 - 实战 ReAct: SerpAPI + LLM-MATH

复习: Model I/O + Chain + Memory



框架原生的数据处理流 Data Connection





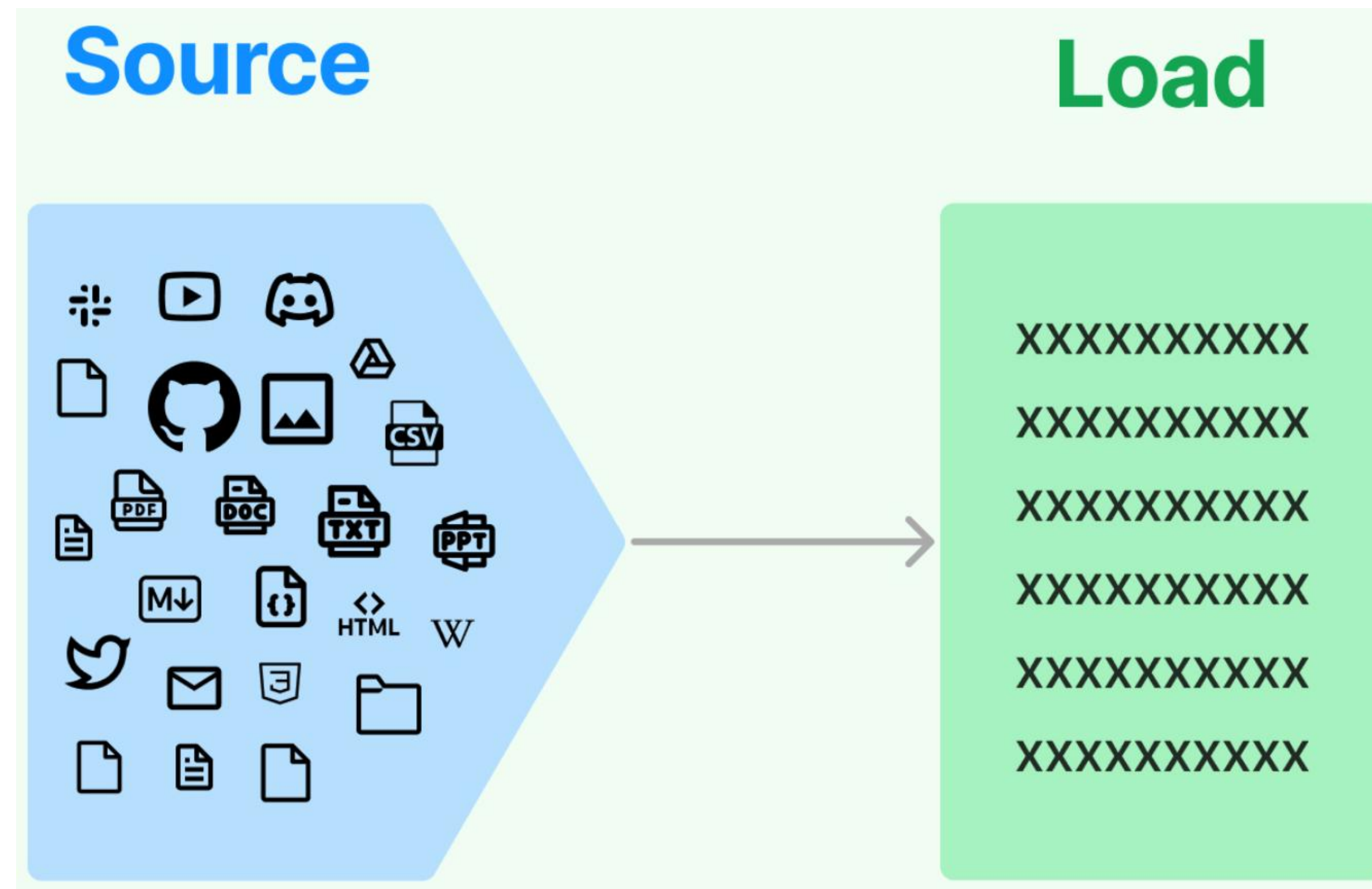
Document Loaders: 快速接入各类数据源

典型数据格式:

- CSV
- HTML
- JSON
- Markdown
- PDF
- File Directory
- ...

部分典型数据源:

- ArXiv
- BiliBili
- Discord
- Figma
- GitHub
- Reddit
- TensorFlow Datasets
- ...



BaseLoader and Document Class

```
"""Base loader class."""

from abc import ABC, abstractmethod
from typing import List, Optional

from langchain.docstore.document import Document
from langchain.text_splitter import RecursiveCharacterTextSplitter, TextSplitter


class BaseLoader(ABC):
    """Base loader class."""

    @abstractmethod
    def load(self) -> List[Document]:
        """Load data into document objects."""

    def load_and_split(
        self, text_splitter: Optional[TextSplitter] = None
    ) -> List[Document]:
        """Load documents and split into chunks."""
        if text_splitter is None:
            _text_splitter: TextSplitter = RecursiveCharacterTextSplitter()
        else:
            _text_splitter = text_splitter
        docs = self.load()
        return _text_splitter.split_documents(docs)
```

```
class Document(BaseModel):
    """Interface for interacting with a document."""

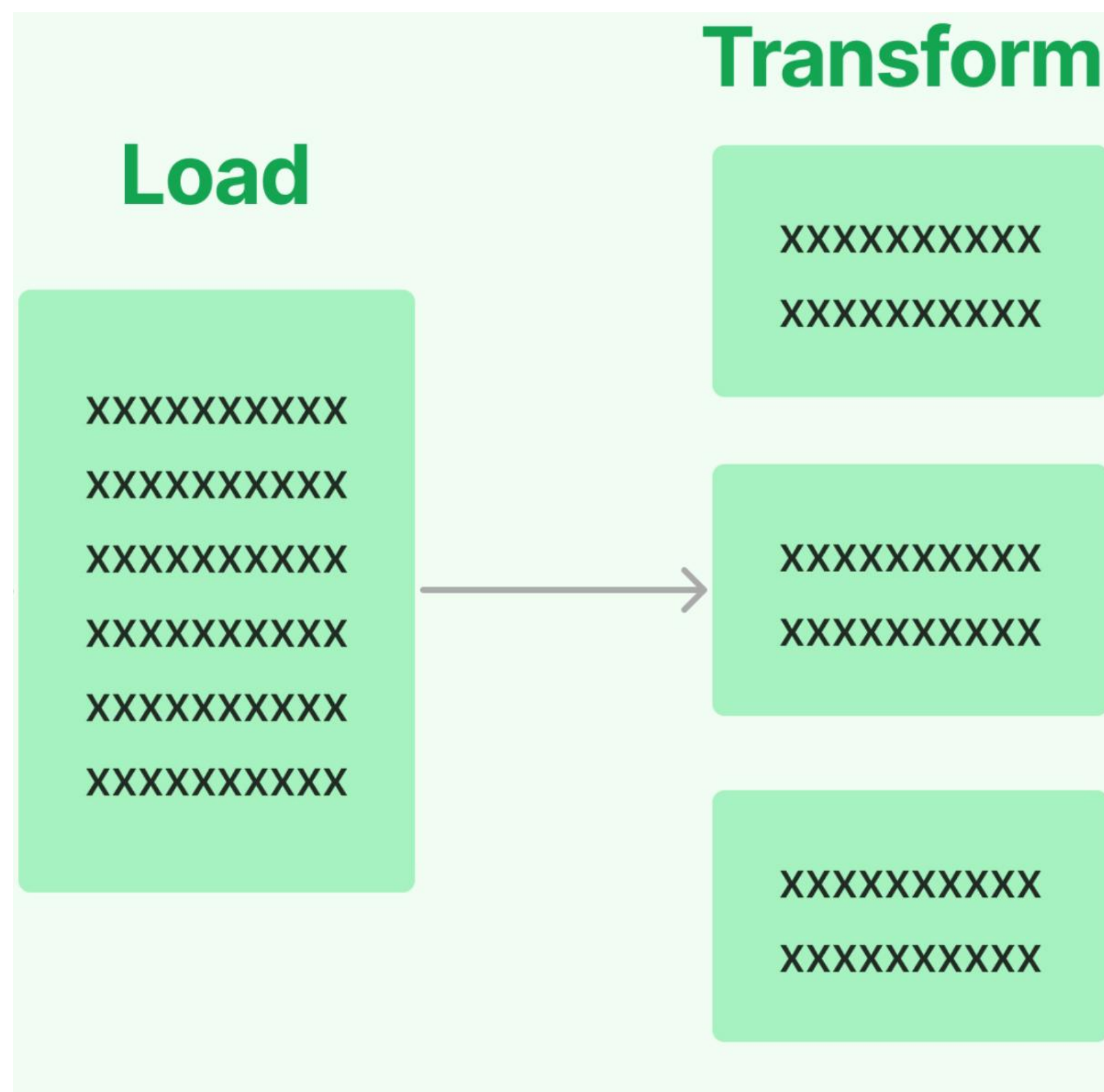
    page_content: str
    lookup_str: str = ""
    lookup_index = 0
    metadata: dict = Field(default_factory=dict)

    @property
    def paragraphs(self) -> List[str]:
        """Paragraphs of the page."""
        return self.page_content.split("\n\n")

    @property
    def summary(self) -> str:
        """Summary of the page (the first paragraph)."""
        return self.paragraphs[0]

    def lookup(self, string: str) -> str:
        """Lookup a term in the page, imitating cmd-F functionality."""
        if string.lower() != self.lookup_str:
            self.lookup_str = string.lower()
            self.lookup_index = 0
        else:
            self.lookup_index += 1
        lookups = [p for p in self.paragraphs if self.lookup_str in p.lower()]
        if len(lookups) == 0:
            return "No Results"
        elif self.lookup_index >= len(lookups):
            return "No More Results"
        else:
            result_prefix = f"(Result {self.lookup_index + 1}/{len(lookups)})"
            return f"{result_prefix} {lookups[self.lookup_index]}"
```

Document Transformers: 原生数据转换模块



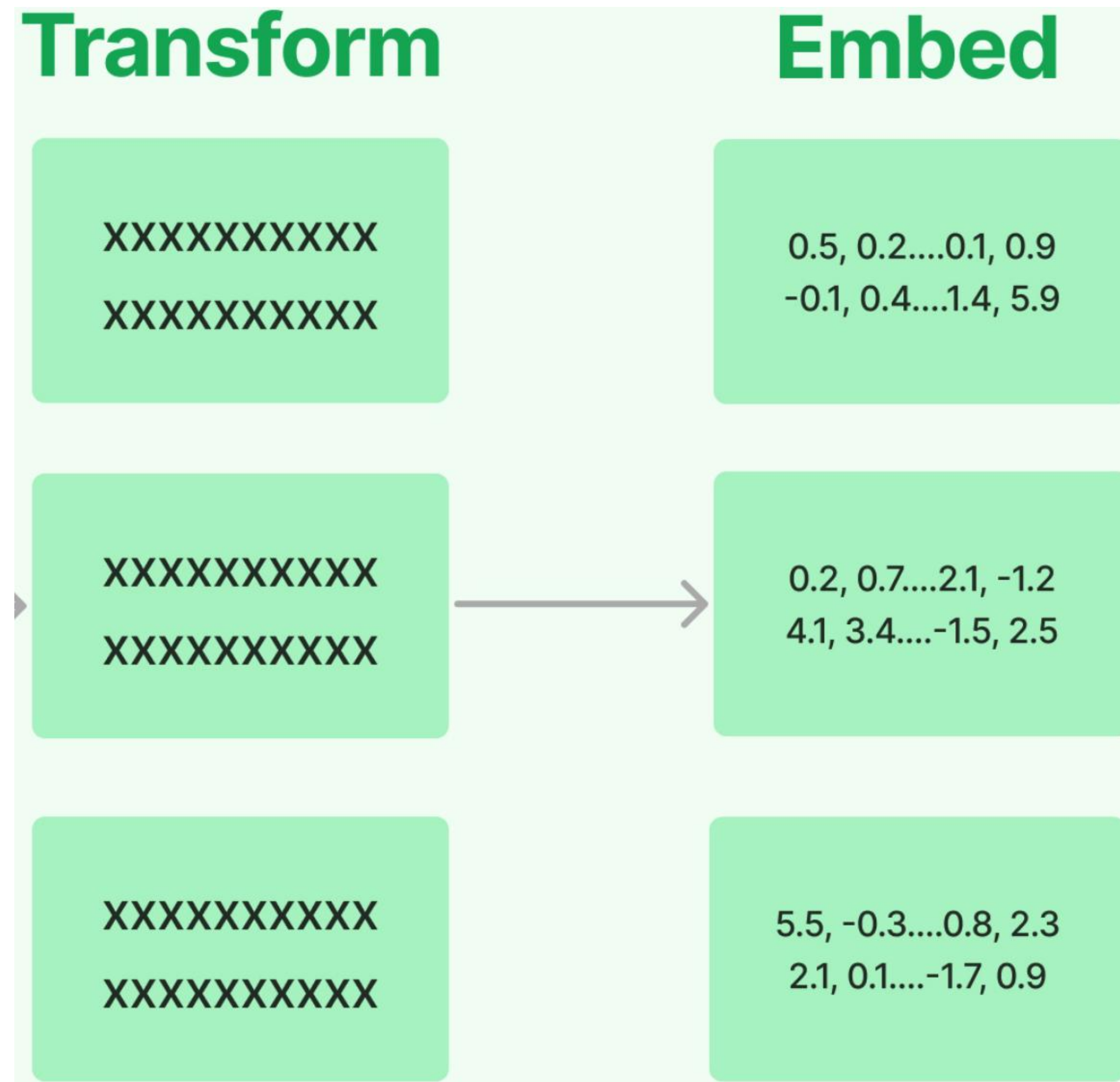
A grid of document transformer tools, each represented by a dark gray box with a document icon, a title, and a brief description:

- Beautiful Soup**: Beautiful Soup offers fine-grained control over HTML cont...
- Doctran Extract Properties**: We can extract useful features of documents using the Do...
- Doctran Interrogate Documents**: Documents used in a vector store knowledge base are typi...
- Doctran Translate Documents**: Comparing documents through embeddings has the benefi...
- html2text**: html2text is a Python script that converts a page of HTML i...
- Nuclia Understanding API docume...**: Nuclia automatically indexes your unstructured data from a...
- OpenAI Functions Metadata Tagger**: It can often be useful to tag ingested documents with struc...

Text Embedding Models:

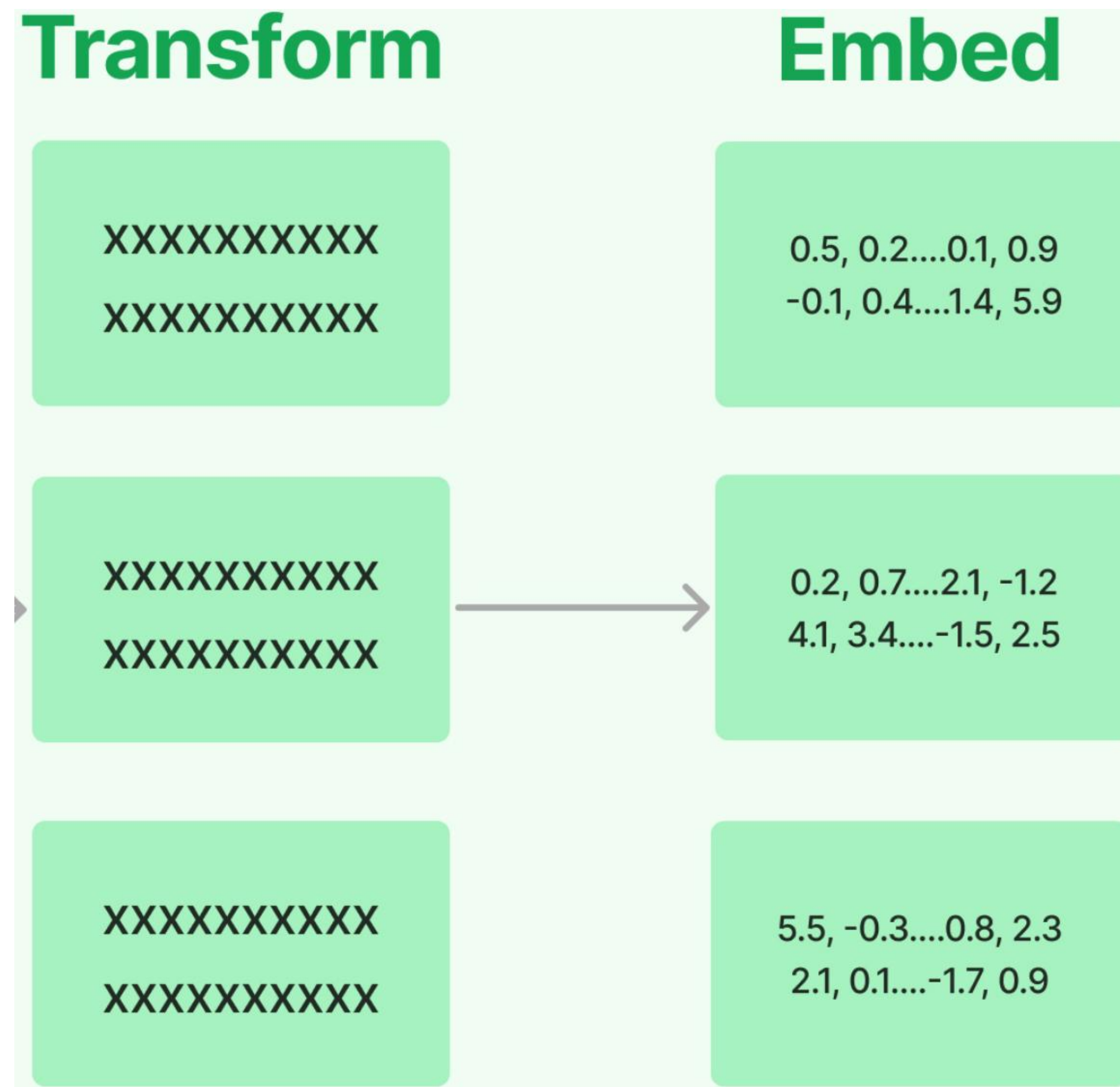
文本嵌入模型

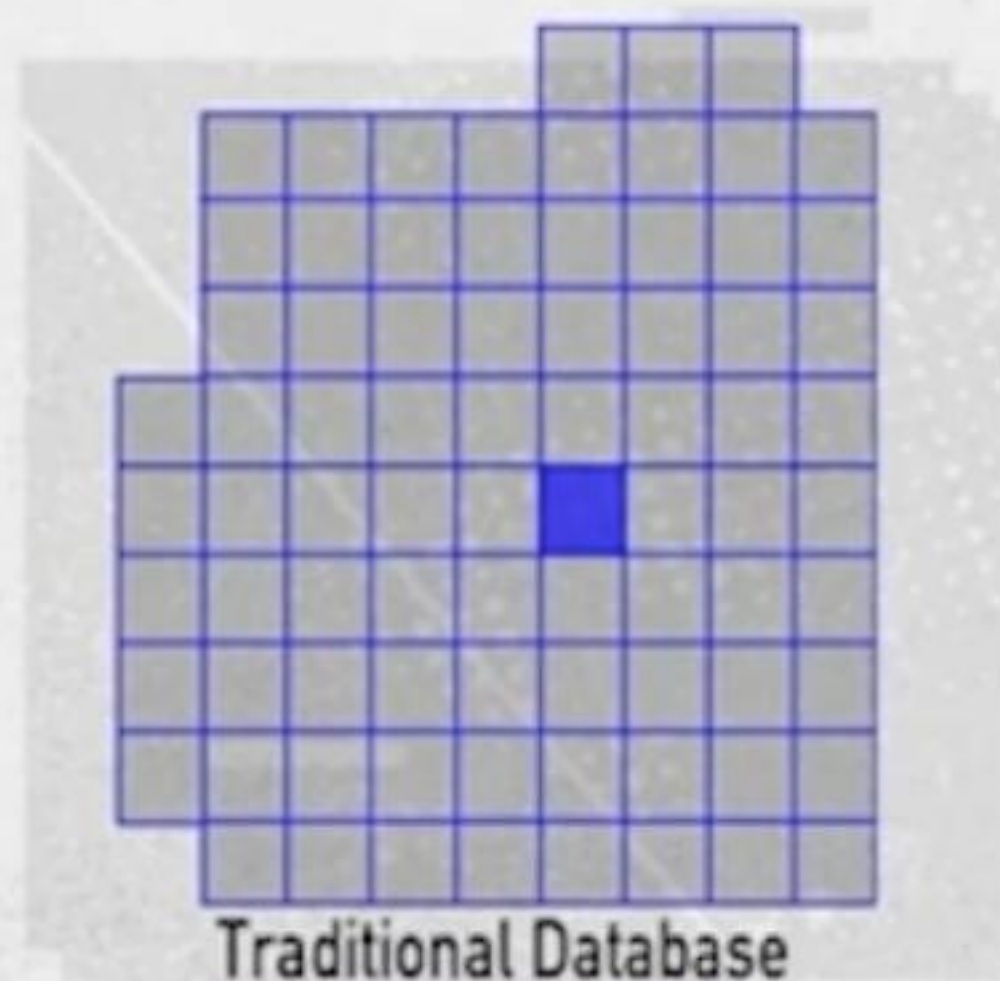
AwaEmbedding
 Aleph Alpha
 AzureOpenAI
 Bedrock Embeddings
 BGE Hugging Face Embeddings
 Clarifai
 Cohere
 DashScope
 DeepInfra
 EDEN AI
 Elasticsearch
 Embaas
 Fake Embeddings
 Google Cloud Platform Vertex AI PaLM
 GPT4All
 Hugging Face Hub



Text Embedding Models: 文本嵌入模型

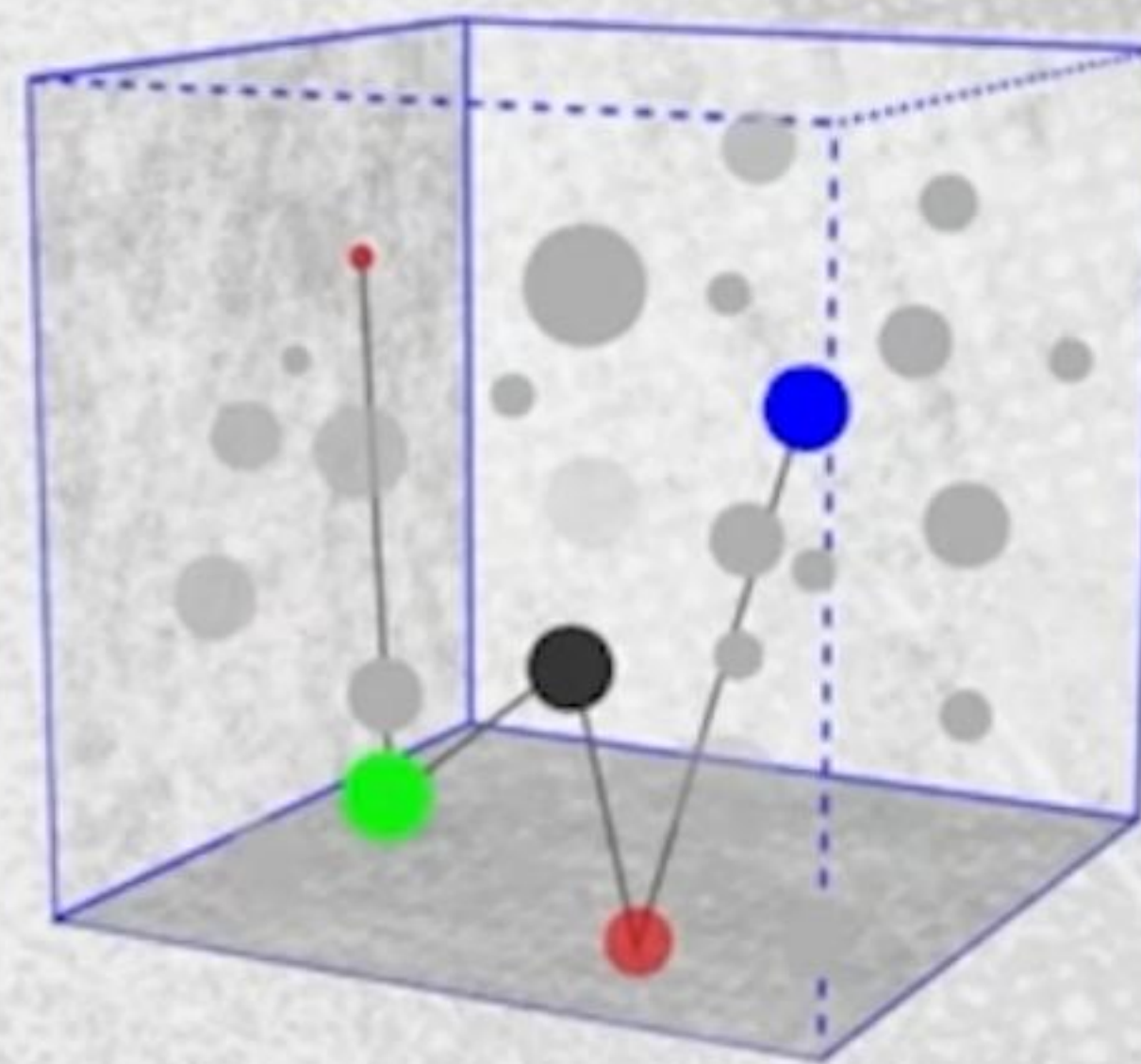
InstructEmbeddings
Jina
Llama-cpp
LocalAI
MiniMax
ModelScope
MosaicML embeddings
NLP Cloud
OpenAI
SageMaker Endpoint Embeddings
Self Hosted Embeddings
Sentence Transformers Embeddings
Spacy Embedding
TensorflowHub
Xorbits inference (Xinference)





Traditional Database

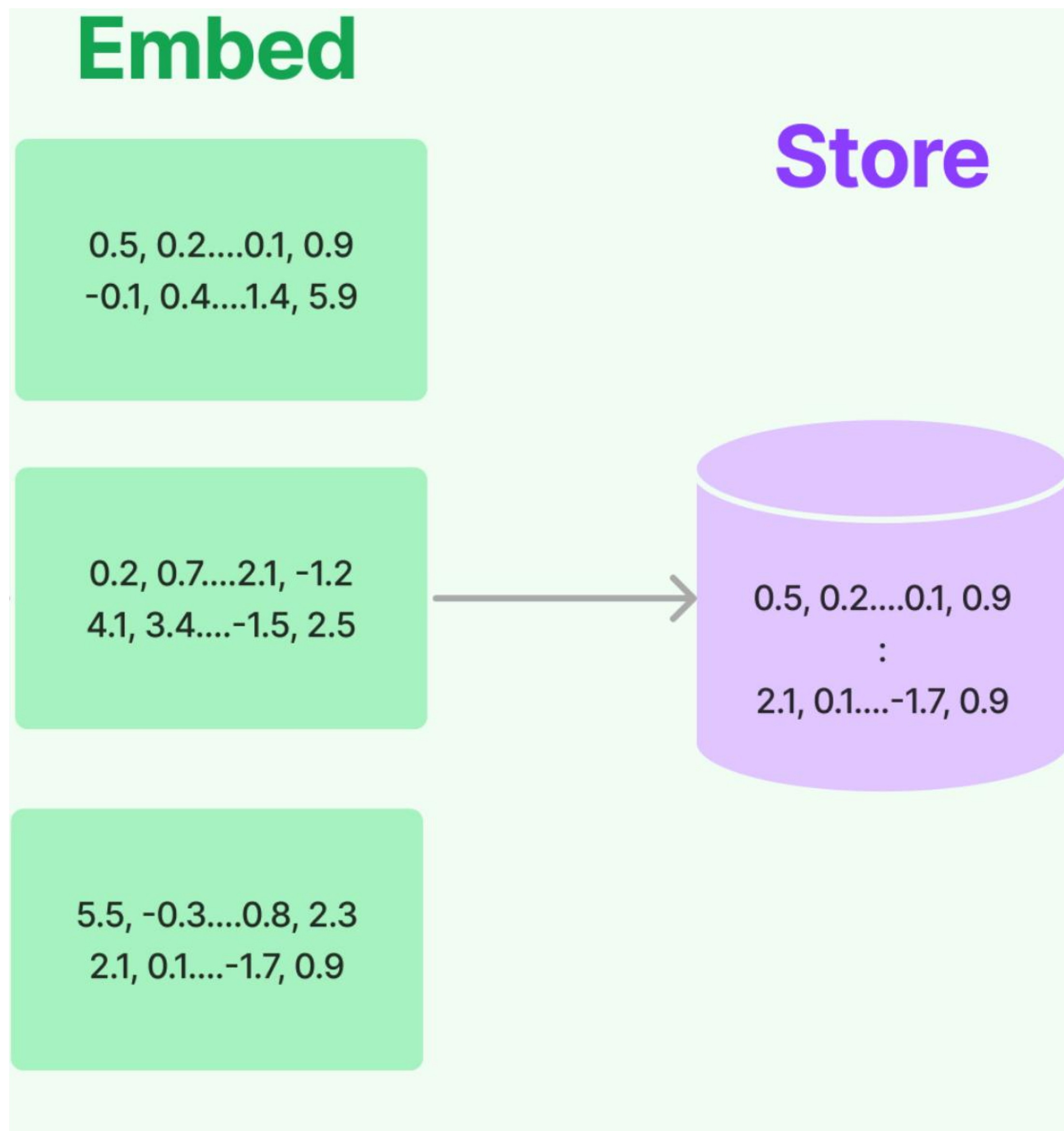
传统数据库



Vector Database

向量数据库

业内首个AI原生的向量数据库



Vector Stores: 向量数据库

- Alibaba Cloud OpenSearch
- AnalyticDB
- Annoy
- Atlas
- AwaDB
- Azure Cognitive Search
- BagelDB
- Cassandra
- **Chroma**
- Clarifai
- ClickHouse Vector Search
- Activeloop's Deep Lake
- Dingo
- DocArrayHnswSearch
- DocArrayInMemorySearch
- ElasticSearch
- FAISS
- Hologres
- LanceDB
- Marqo
- MatchingEngine
- Meilisearch
- Milvus
- MongoDB Atlas
- MyScale
- OpenSearch
- pg_embedding

Embed

0.5, 0.2....0.1, 0.9
-0.1, 0.4....1.4, 5.9

0.2, 0.7....2.1, -1.2
4.1, 3.4....-1.5, 2.5

5.5, -0.3....0.8, 2.3
2.1, 0.1....-1.7, 0.9

Store

0.5, 0.2....0.1, 0.9
:
2.1, 0.1....-1.7, 0.9

Vector Stores: 向量数据库

- **PGVector**
- **Pinecone**
- Qdrant
- Redis
- Rockset
- ScaNN
- SingleStoreDB
- scikit-learn
- StarRocks
- Supabase (Postgres)
- Tair
- Tigris
- Typesense
- USearch
- Vectara
- Weaviate
- Xata
- Zilliz

Vector Stores

1. Load Source Data



Load, Transform, Embed

Vector Store



2. Query Vector Store

Embed

5.5, -0.3...
2.1, 0.1

XXXXXXXXXXXXX
XXXXXXXXXXXXX

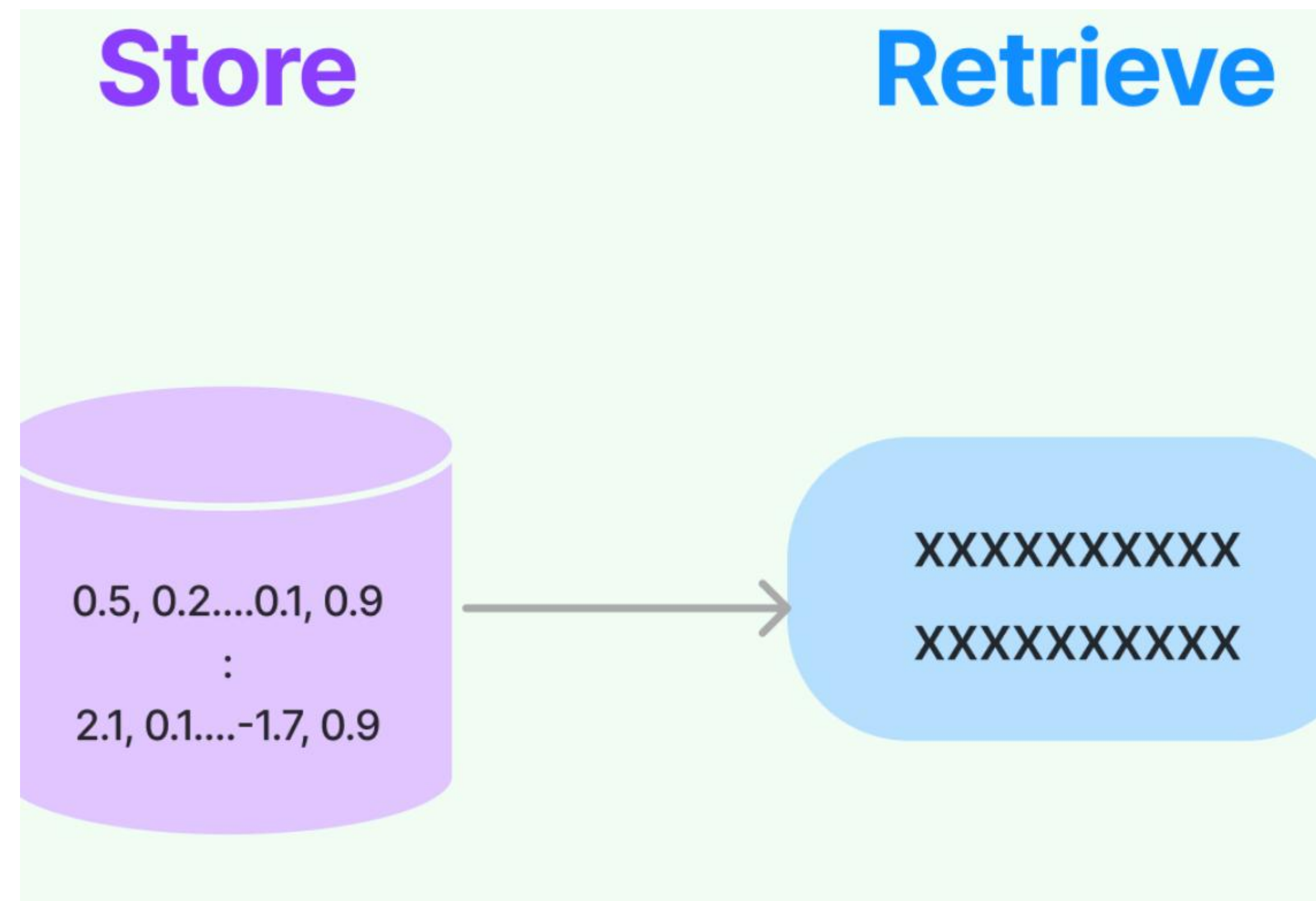
XXXXXXXXXXXXX
XXXXXXXXXXXXX

3. Retrieve 'most similar'

向量数据库最佳实践

Retrieves: 数据检索器

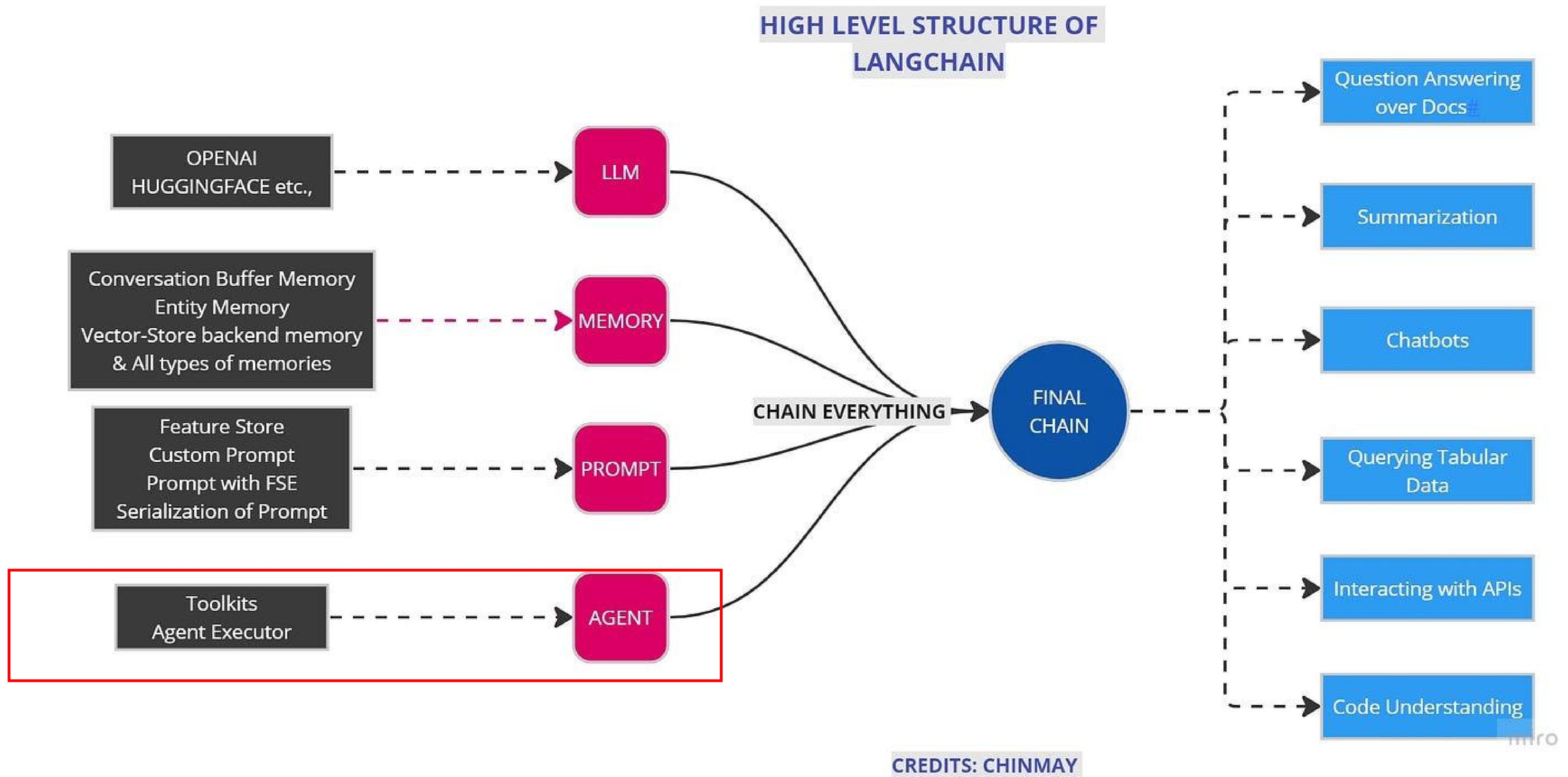
- Amazon Kendra
- Arxiv
- Azure Cognitive Search
- BM25
- Chaindesk
- ChatGPT Plugin
- Cohere Reranker
- DocArray Retriever
- Elasticsearch BM25
- Google Cloud Enterprise Search
- kNN
- LOTR (Merger Retriever)
- Metal
- Pinecone Hybrid Search
- PubMed
- RePhraseQueryRetriever
- SVM
- TF-IDF
- Vespa
- Weaviate Hybrid Search
- Wikipedia
- Zep



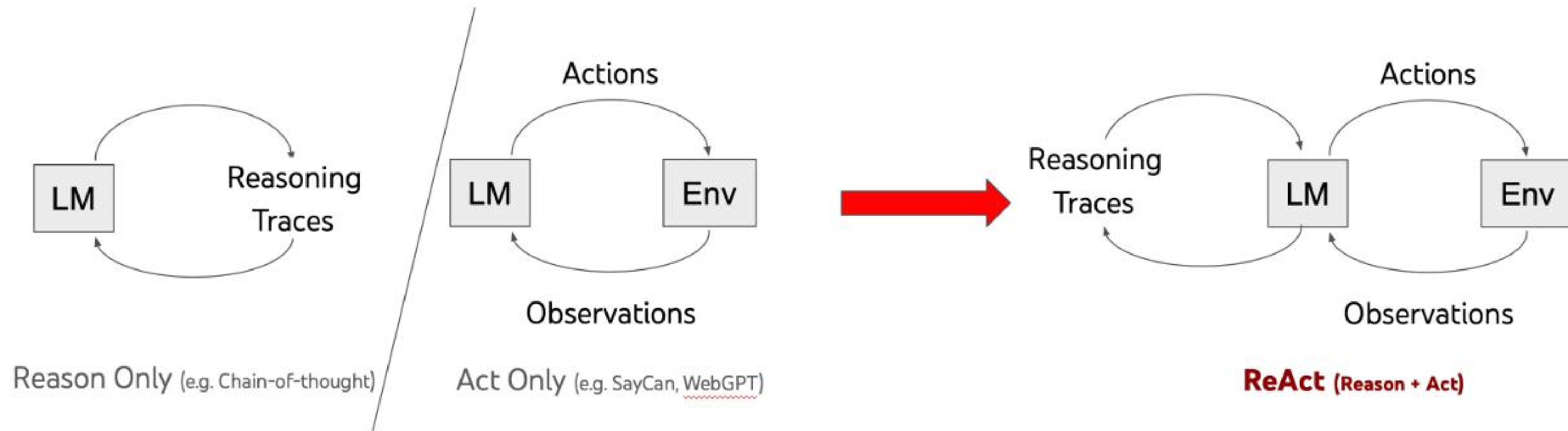
DEMO

Agent

Agent: 构建复杂应用的代理系统



ReAct: Reasoning + Acting



ReAct Prompt 由 few-shot task-solving trajectories 组成，包括人工编写的文本推理过程和动作，以及对动作的环境观察.

ReAct Prompt 设计直观灵活，并在各种任务上实现了最先进的少样本性能，从QA到在线购物

Prompting Method	HotpotQA (<u>question answering</u> , exact match, 6-shot)	FEVER (<u>fact verification</u> , accuracy, 3-shot)	ALFWorld (<u>text-based game</u> , succ rate, 2-shot)	WebShop (<u>web interaction</u> , succ rate, 1-shot)
Standard (IO)	28.7	57.1	N/A (cannot act)	N/A (cannot act)
Reason-only (CoT)	29.4	56.3	N/A (cannot act)	N/A (cannot act)
Act-only	25.7	58.9	45	30.1
Best ReAct method	35.1	64.6	71	40
Supervised/Imitation Learning SoTA	67.5 (140k samples)	89.5 (90k samples)	37 (100k samples)	29.1 (90k samples)

ReAct 在获取新数据方面的优势 (HotpotQA 示例)

Reason-only baseline （即思维链） 由于没有与外部环境接触以获取和更新知识， 而且必须依赖有限的内部知识， 因此容易受到错误信息（红色标记） 的影响。

Act-only baseline 缺乏推理能力方面问题， 在这种情况下， 尽管具有与ReAct相同的行动和观察， 但无法综合得出最终答案。
相比之下， ReAct通过可解释且真实可信的轨迹来解决任务。

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

Standard

Answer: iPod



Reason only

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch



Act only

Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced...
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ...
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software ...
Act 4: Finish[yes]



ReAct

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: Search[Apple Remote]

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: Search[Front Row]

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: Search[Front Row (software)]

Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

Act 4: Finish[keyboard function keys]



LangChain 实例

```
[12]: self_ask_with_search = initialize_agent(
        tools, llm, agent=AgentType.SELF_ASK_WITH_SEARCH, verbose=True
    )
self_ask_with_search.run(
    "2023年大运会举办地在哪里?"
)
```

ReAct

> Entering new AgentExecutor chain...

Yes.

Follow up: 什么是2023年大运会?

Intermediate answer: The 2023 FISU Summer World University Games, officially known as the 2023 Summer Universiade or the XXXII Summer World University Games, and commonly known as Yekaterinburg 2023 or E katerinburg 2023, ...

will be the 32nd edition of the Summer Universiade, an international multi-sport event organised for university athletes by the International University Sports Federation (FISU).

Follow up: 2023年大运会的举办地在哪里?

Intermediate answer: #成都大运会#东安湖体育中心9月28日, 成都大运会组委会在其官网公布了第31届世界大学生夏季运动会竞赛总日程, 其中开幕式将于2023年7月28日举行, 闭幕式将于8月8日举行。 你期待成都大运会么?

So the final answer is: 成都

> Finished chain.

```
[12]: '成都'
```

```
[14]: print(llm("2023年大运会举办地在哪里?"))
```

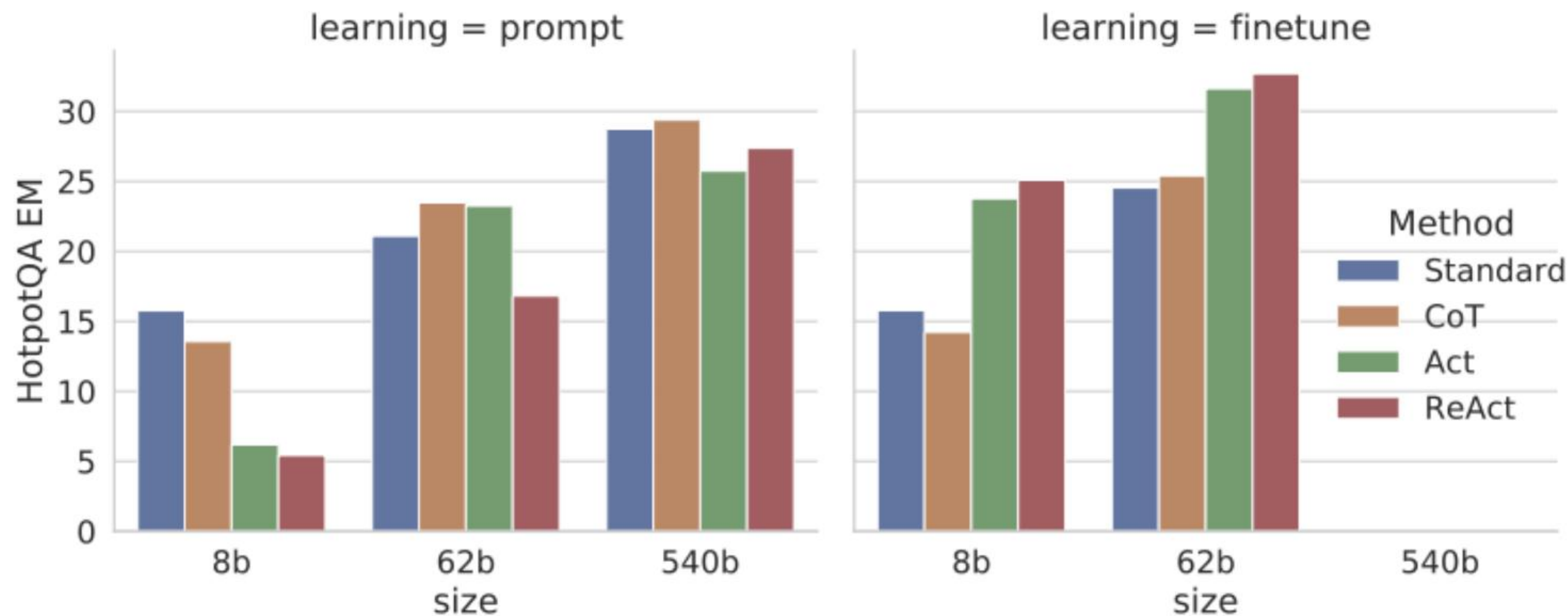
2023年大运会将在日本东京举办。

Reason-only

ReAct 在微调方面的优势

使用 ReAct 提示轨迹在 HotpotQA 上进行初始微调的结果表明:

- (1) ReAct 是各种模型规模中最好的微调方式;
- (2) ReAct 微调的较小模型胜过了被提示 (prompted) 的更大模型。

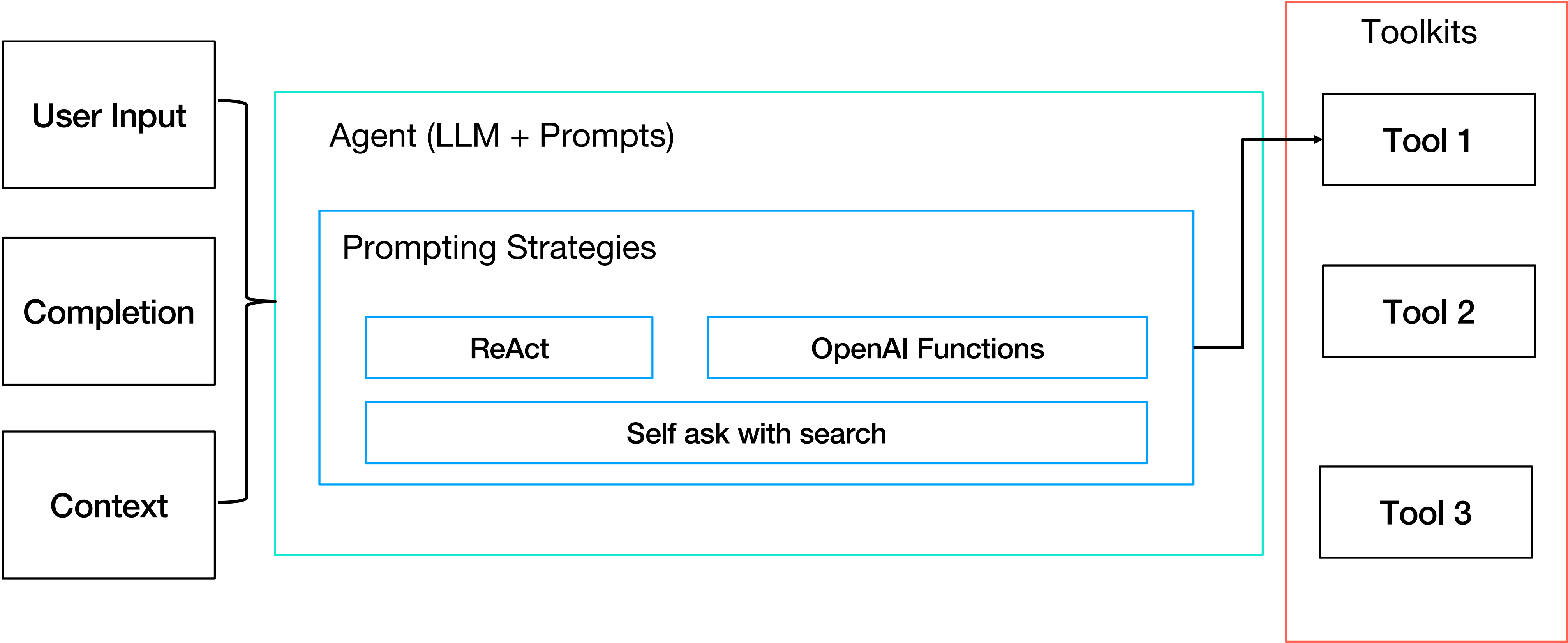
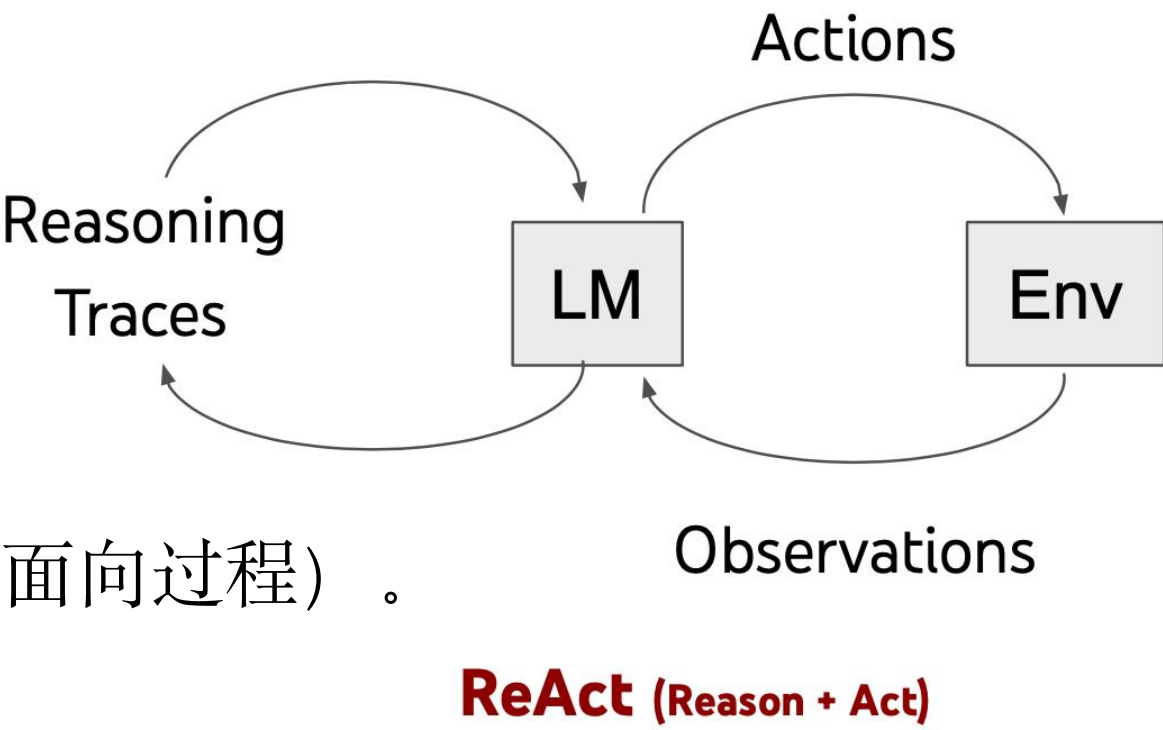


LangChain Agents

LangChain Agents 设计原理

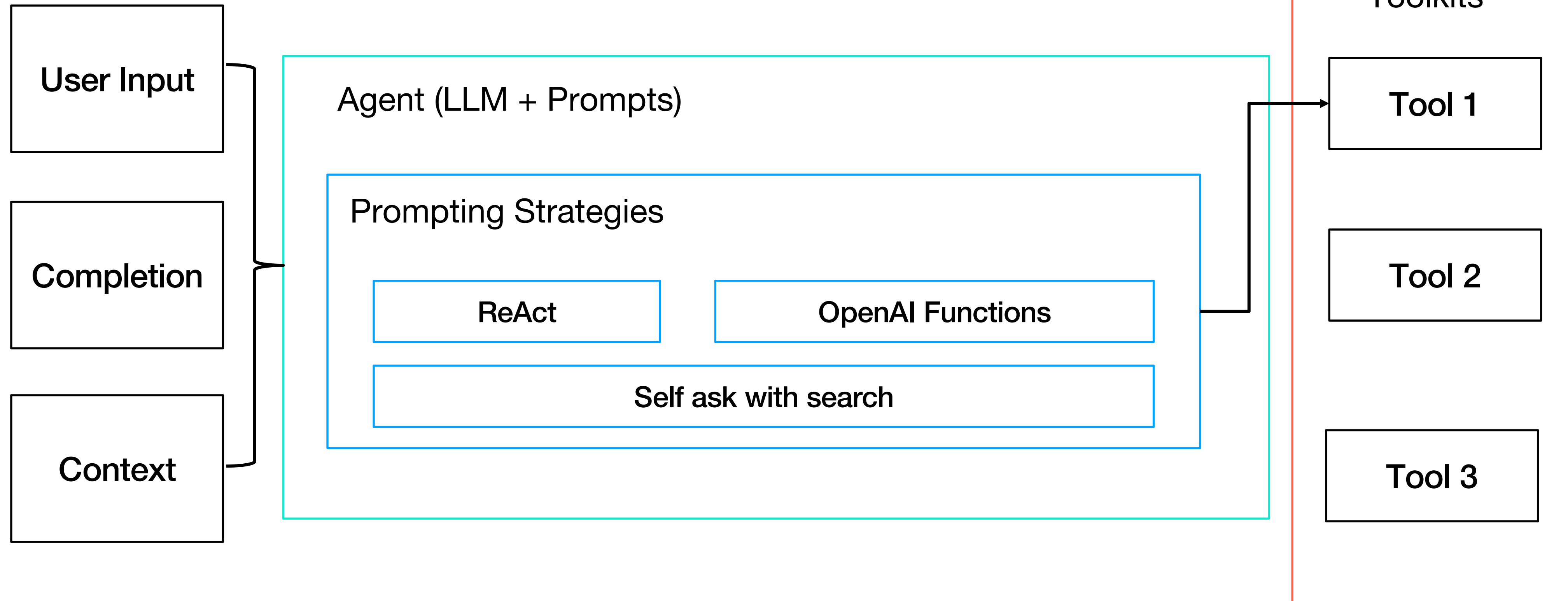
代理的核心思想是使用LLM来选择一系列要执行的动作。

- 在链式结构（Chains）中，一系列动作执行是硬编码的（ SequentialChain 和 RouterChain 也仅实现了面向过程）。
- 在代理（Agents）中，语言模型被用作推理引擎，以确定应该采取哪些动作以及执行顺序。



```
next_action = agent.get_action(...)
while next_action != AgentFinish:
    observation = run(next_action)
    next_action = agent.get_action(..., next_action, observation)
return next_action
```

AgentExecutor (Runtime)

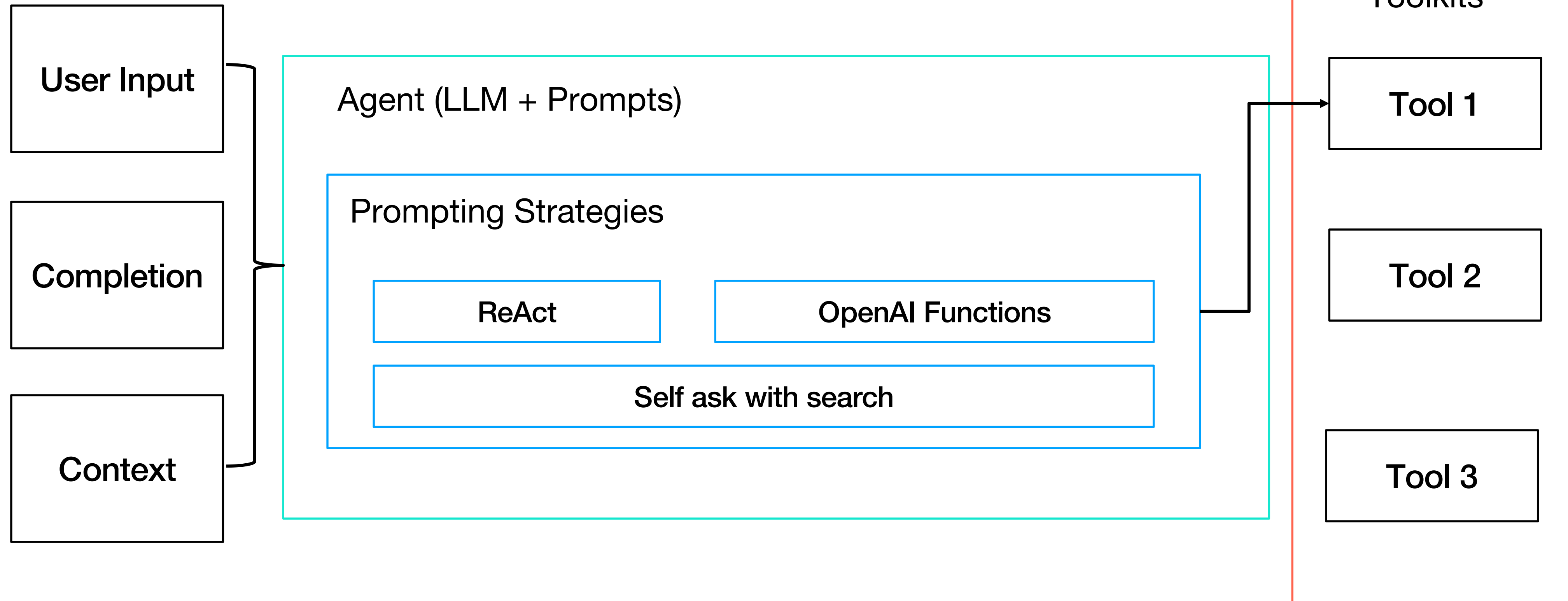


LangChain Agents 设计原理

- Plan-and-execute Agent
- BabyAGI
- AutoGPT

```
next_action = agent.get_action(...)
while next_action != AgentFinish:
    observation = run(next_action)
    next_action = agent.get_action(..., next_action, observation)
return next_action
```

AgentExecutor (Runtime)



实战 Self ask with search

SerpApi

Documentation

Integrations

Features

Pricing

Use cases

Team

FAQ

Contact us

[Sign In](#)

[Register](#)

Google Search API

Search Query

Location

Coffee

Austin, Texas, United States

TEST SEARCH

Coffee

About 2,520,000,000 results (1.13 seconds)

Coffee Shops

- Starbucks**
4.1 ★★★★★ (506) • \$• Coffee shop
600 Congress Ave
Dine-in • Takeout • Delivery
- Houndstooth Coffee**
4.6 ★★★★★ (740) • \$• Coffee shop
401 Congress Ave #100C - In Frost Bank Tower
Dine-in • Takeout • No delivery
- Lucky Lab Coffee**
3.0 ★★★★★ (2) • Cafe
515 Congress Ave - In the Bank of America Financial Center
Dine-in • Takeout • No delivery

[View all](#)

<https://en.wikipedia.org/wiki/Coffee>

Coffee - Wikipedia

Coffee is a brewed drink prepared from roasted coffee beans, the seeds of berries from certain Coffea species. From the coffee fruit, the seeds are ...
Region of origin: Horn of Africa and South Afr... Color: Black, dark brown, light brown, beige
Introduced: 15th century
History · Coffee bean · Coffee preparation · Coffee production

People also ask

- Is coffee good for your health?
- What are the 4 types of coffee?
- Does Austin have good coffee?
- What does coffee do to your body?

<https://www.coffeebean.com>

Home | The Coffee Bean & Tea Leaf

Never run out of your favorite coffees, teas and powders again with our auto-delivery subscription. Select how often your products arrive, pause and cancel ...

<https://www.healthline.com/nutrition/top-13-eviden...>

13 Health Benefits of Coffee, Based on Science - Healthline

Coffee

Drink

Coffee is a brewed drink prepared from roasted coffee beans, the seeds of berries from certain Coffea species. From the coffee fruit, the seeds are separated to produce a stable, raw product: unroasted green coffee. [Wikipedia](#)

Patron saint: Saint Drogo of Sebourg
[medievaltimes.com](#)

Nutrition Facts
Coffee " "
Sources include: [USDA](#)

Amount Per 1 cup (8 fl oz) (237 g)	% Daily Value*
Calories 1	
Total Fat 0 g	0%
Saturated fat 0 g	0%
Trans fat regulation 0 g	
Cholesterol 0 mg	0%
Sodium 5 mg	0%
Potassium 116 mg	3%
Total Carbohydrate 0 g	0%
Dietary fiber 0 g	0%
Sugar 0 g	
Protein 0.3 g	0%
Caffeine 95 mg	
Vitamin C 0%	Calcium 0%
Iron 0%	Vitamin D 0%
Vitamin B6 0%	Cobalamin 0%
Magnesium 1%	

*Percent Daily Values are based on a 2,000 calorie diet. Your daily values may be higher or lower depending on your calorie needs.

Chicory coffee [View 5+ more](#)

```
{
  "search_metadata": {
    "id": "6165916694c6c7025deef5ab",
    "status": "Success",
    "json_endpoint": "https://serpapi.com/searches/87fa874d05a7fcc5/6165916694c6c7025deef5ab.js",
    "created_at": "2021-10-12 13:45:10 UTC",
    "processed_at": "2021-10-12 13:45:11 UTC",
    "google_url": "https://www.google.com/search?q=Coffee&oq=Coffee&uule=w+CAIQICIAQXVzdGluLF...",
    "raw_html_file": "https://serpapi.com/searches/87fa874d05a7fcc5/6165916694c6c7025deef5ab.h...",
    "total_time_taken": 1.85
  },
  "search_parameters": {
    "engine": "google",
    "q": "Coffee",
    "location_requested": "Austin, Texas, United States",
    "location_used": "Austin,Texas,United States",
    "google_domain": "google.com",
    "hl": "en",
    "gl": "us",
    "device": "desktop"
  },
  "search_information": {
    "organic_results_state": "Results for exact spelling",
    "query_displayed": "Coffee",
    "total_results": 2520000000,
    "time_taken_displayed": 1.13
  },
  "recipes_results": [
    {
      "title": "20 Great Coffee Drinks",
      "link": "https://www.acouplecooks.com/coffee-drinks/",
      "source": "A Couple Cooks",
      "total_time": "5 min",
      "ingredients": [
        "Moka pot",
        "dairy free"
      ]
    },
    {
      "title": "How to Make Iced Coffee",
      "link": "https://www.recipegirl.com/how-to-make-iced-coffee/",
      "source": "Recipe Girl",
      "total_time": "5 min",
      "ingredients": [
        "Simple syrup",
        "milk"
      ]
    }
  ]
}
```

Play with more parameters and search engines [↗](#)

Google + GPT

```
[ ]: import os

os.environ["SERPAPI_API_KEY"] = "4608b4e90f7db9e68bb045f3feee45725b"

[10]: from langchain import OpenAI, SerpAPIWrapper
      from langchain.agents import initialize_agent, Tool
      from langchain.agents import AgentType

      llm = OpenAI(temperature=0)

[11]: search = SerpAPIWrapper()
      tools = [
          Tool(
              name="Intermediate Answer",
              func=search.run,
              description="useful for when you need to ask with search",
          )
      ]

[12]: self_ask_with_search = initialize_agent(
      tools, llm, agent=AgentType.SELF_ASK_WITH_SEARCH, verbose=True
      )
      self_ask_with_search.run(
          "2023年大运会举办地在哪里？"
      )
```

> Entering new AgentExecutor chain...

Yes.

Follow up: 什么是2023年大运会?

Intermediate answer: The 2023 FISU Summer World University Games, officially known as the 2023 Summer Universiade or the XXXII Summer World University Games, and commonly known as Yekaterinburg 2023 or Ekaterinburg 2023, ...

will be the 32nd edition of the Summer Universiade, an international multi-sport event organised for university athletes by the International University Sports Federation (FISU).

Follow up: 2023年大运会的举办地在哪里?

Intermediate answer: #成都大运会#东安湖体育中心9月28日, 成都大运会组委会在其官网公布了第31届世界大学生夏季运动会竞赛总日程, 其中开幕式将于2023年7月28日举行, 闭幕式将于8月8日举行。 你期待成都大运会么?

So the final answer is: 成都

> Finished chain.

[12]: '成都'

实战 **ReAct**

ReAct

LangChain 核心模块 Agent - ReAct

ReAct 核心思想是 推理+操作，本示例以 Google Search 和 LLM Math 作为可选操作集合 (toolkits)，实现 ReAct 功能。

```
[4]: import os

os.environ["SERPAPI_API_KEY"] = "90f7db9e68bb045f3feee45725b"

[5]: from langchain.llms import OpenAI

llm = OpenAI(temperature=0)

[6]: from langchain.agents import load_tools
from langchain.agents import initialize_agent
from langchain.agents import AgentType

tools = load_tools(["serpapi", "llm-math"], llm=llm)
```

使用语言模型 (GPT-3)

```
[7]: agent = initialize_agent(tools, llm, agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION, verbose=True)
```

```
• [8]: agent.run("谁是莱昂纳多·迪卡普里奥的女朋友？她现在年龄的0.43次方是多少？")
```

```
> Entering new AgentExecutor chain...
I need to find out who Leonardo DiCaprio's girlfriend is and then calculate her age to the 0.43 power.
Action: Search
Action Input: "Leonardo DiCaprio girlfriend"
Observation: Camila Morrone It's believed Morrone and DiCaprio began dating in 2017.
Thought: I need to find out how old Camila Morrone is.
Action: Search
Action Input: "Camila Morrone age"
Observation: 26 years
Thought: I need to calculate 26 to the 0.43 power.
Action: Calculator
Action Input: 26^0.43
Observation: Answer: 4.059182145592686
Thought: I now know the final answer.
Final Answer: Camila Morrone is Leonardo DiCaprio's girlfriend and her age to the 0.43 power is 4.059182145592686.
```

```
> Finished chain.
```

```
[8]: "Camila Morrone is Leonardo DiCaprio's girlfriend and her age to the 0.43 power is 4.059182145592686."
```

Tools

Tools

Alpha Vantage
Apify
ArXiv API Tool
AWS Lambda API
Shell Tool
Bing Search
Brave Search
ChatGPT Plugins
Dall-E Image Generator
DataForSeo API Wrapper
DuckDuckGo Search
File System Tools
Golden Query
Google Places
Google Search
Google Serper API
Gradio Tools
GraphQL tool
huggingface_tools
Human as a tool
IFTTT WebHooks
Lemon AI NLP Workflow Automation
Metaphor Search
Nuclia Understanding API tool
OpenWeatherMap API

```
from langchain.agents import load_tools
from langchain.agents import initialize_agent

tools = load_tools(['dalle-image-generator'])
agent = initialize_agent(tools, llm, agent="zero-shot-react-description", verbose=True)
output = agent.run("Create an image of a halloween night at a haunted museum")
```

```
# Requires transformers>=4.29.0 and huggingface_hub>=0.14.1
pip install --upgrade transformers huggingface_hub > /dev/null
```

```
from langchain.agents import load_huggingface_tool

tool = load_huggingface_tool("lysandre/hf-model-downloads")

print(f"{tool.name}: {tool.description}")
```

```
from langchain.tools import WikipediaQueryRun
from langchain.utilities import WikipediaAPIWrapper
```

API Reference:

- `WikipediaQueryRun` from `langchain.tools`
- `WikipediaAPIWrapper` from `langchain.utilities`

```
wikipedia = WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())
```

```
wikipedia.run("HUNTER X HUNTER")
```

```
'Page: Hunter × Hunter\nSummary: Hunter × Hunter (stylized as HUNTER×HUNTER and pronounced "hunter
```

Tools

OpenWeatherMap API
PubMed Tool
Requests
SceneXplain
Search Tools
SearxNG Search API
SerpAPI
SQL Database Chain
Twilio
Wikipedia
Wolfram Alpha
YouTubeSearchTool
Zapier Natural Language Actions API

```
from langchain.agents import load_tools
from langchain.agents import initialize_agent

tools = load_tools(['dalle-image-generator'])
agent = initialize_agent(tools, llm, agent="zero-shot-react-description", verbose=True)
output = agent.run("Create an image of a halloween night at a haunted museum")
```

```
# Requires transformers>=4.29.0 and huggingface_hub>=0.14.1
pip install --upgrade transformers huggingface_hub > /dev/null
```

```
from langchain.agents import load_huggingface_tool

tool = load_huggingface_tool("lysandre/hf-model-downloads")

print(f"{tool.name}: {tool.description}")
```

```
from langchain.tools import WikipediaQueryRun
from langchain.utilities import WikipediaAPIWrapper
```

API Reference:

- `WikipediaQueryRun` from `langchain.tools`
- `WikipediaAPIWrapper` from `langchain.utilities`

```
wikipedia = WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())
```

```
wikipedia.run("HUNTER X HUNTER")
```

```
'Page: Hunter × Hunter\nSummary: Hunter × Hunter (stylized as HUNTER×HUNTER and pronounced "hunter
```

DEMO

课程项目：GitHub openai-quickstart

openai-quickstartPublic

UnpinUnwatch10Fork145Starred306

main1 branch0 tagsGo to fileAdd fileCode

DjangoPeng Merge branch 'main' of github.com:DjangoPeng/openai-quickstart int...6bd7e132 minutes ago57 commits

chatgpt-plugins	add initial version of chatgpt plugins	2 minutes ago
docs	make layout more readable	last week
openai-translator	Merge branch 'main' of github.com:DjangoPeng/openai-quickstart in...	2 minutes ago
openai_api	Remove fine_food_reviews_with_embeddings_1k_demo.csv	last week
.gitignore	update embedding demo	2 weeks ago
LICENSE	Initial commit	2 weeks ago
README-CN.md	fix: correct file path of pdfplumber.ipynb on readme	yesterday
README.md	fix: correct file path of pdfplumber.ipynb on readme	yesterday

README.md

OpenAI Quickstart

English | 中文

This project is designed as a one-stop learning resource for anyone interested in large language models and their application in Artificial Intelligence Governance and Control (AIGC) scenarios. By providing theoretical foundations, development basics, and hands-on examples, this project offers comprehensive guidance on these cutting-edge topics.

Features

About

A comprehensive guide to understanding and implementing large language models with hands-on examples using LangChain for AIGC applications.

- Readme
- Apache-2.0 license
- Activity
- 306 stars
- 10 watching
- 145 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors8



<https://github.com/DjangoPeng/openai-quickstart>